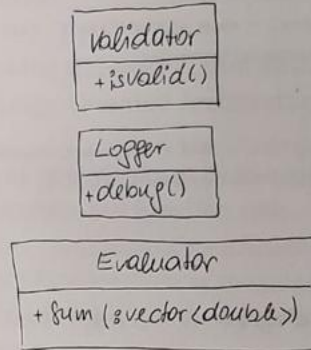


SDA EXAM 5.02.2025

1) Given the following class diagram: [5 + 5 points]



- Extend the **class diagram** to a **Facade Pattern**. Add the "Calculator" facade class and the "Client". Add a method called "calcSum(:vector<double>)" to the "Calculator" class.
- The facade method "calcSum(:vector<double>)" calls "debug()", "isValid()", and "sum()" in sequence. Draw a **sequence diagram** that shows the processing of the method "calcSum(:vector<double>)".

Q1) Facade pattern

1(a) Extend to Facade (class diagram idea)

You already have:

- **Validator** with +isValid()
- **Logger** with +debug()
- **Evaluator** with +sum(vector<double>)

Add:

- **Calculator** (Facade)
 - +calcSum(vector<double>) : double
 - has (uses/owns) references to Logger, Validator, Evaluator
- **Client**
 - calls the facade only (Client should not call Validator/Logger/Evaluator directly)

Relationships (simple):

- Client --> Calculator
- Calculator --> Logger

- Calculator --> Validator
- Calculator --> Evaluator

1(b) Sequence diagram for `calcSum(vector<double>)`

The method must call in sequence: `debug()`, `isValid()`, `sum()`.

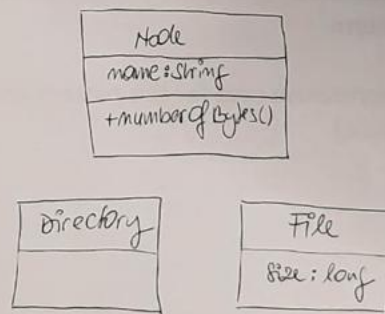
Text form of the sequence:

1. **Client** → **Calculator**: `calcSum(values)`
2. **Calculator** → **Logger**: `debug()`
3. **Calculator** → **Validator**: `isValid()`
 - returns `true/false`
4. **Calculator** → **Evaluator**: `sum(values)` (*only after validation call, and as the sequence says*)
 - returns result (double)
5. **Calculator** → **Client**: return result

If you want to show what happens when invalid, add an **alt** block:

- if `isValid() == false` → return error/0/exception (depends on your design)

2) Given the following class diagram: [4 + 6 points]



- a) Insert the **missing relationships** between the classes to get a **Composite Pattern**.
- b) Draw the **object diagram** (including **attribute values**) for the given test case.

```

35  ✓ TEST(CompositeTestGroup, FileSystemTest)
36  {
37      auto root = make_shared<Directory>("/");
38      auto home = make_shared<Directory>("home");
39      root->addNode(home);
40
41      auto teini = make_shared<Directory>("teini");
42      root->addNode(teini);
43      auto bashrc = make_shared<File>(".bashrc", 1024);
44      teini->addNode(bashrc);
45      auto todo = make_shared<File>("TODO.txt", 2048);
46      teini->addNode(todo);
47
48      auto server = make_shared<Directory>("www");
49      root->addNode(server);
50      auto bashrc2 = make_shared<File>(".bashrc", 512);
51      server->addNode(bashrc2);
52
53      // Verify
54      EXPECT_EQ(1024 + 2048 + 512, root->numberOfBytes());
55      EXPECT_EQ(3, root->numberOfFiles());
56  }

```

2) Composite Pattern (File system)

2(a) Insert missing relationships to get Composite

Classes:

- **Node** (base)
 - name : string
 - +numberOfBytes() : long
- **Directory**
- **File**
 - size : long

To build **Composite**:

✓ Inheritance

- Directory **extends** Node
- File **extends** Node

✓ Composition (Directory contains Nodes)

- Directory contains **0..*** children of type Node
 - e.g. children : list<Node>
- UML: Directory 1  0..* Node (black diamond)

✓ Method meaning

- File.numberofBytes() = returns its size
- Directory.numberofBytes() = sum of numberofBytes() of all children recursively

(Your test code also uses numberofFiles() → typically:

- File.numberofFiles() = 1
- Directory.numberofFiles() = sum(children.numberofFiles()))

2(b) Object diagram for the given test case (with attributes)

From the shown test:

- root = Directory "/"
- home = Directory "home" added to root
- teini = Directory "teini" added to root
 - .bashrc File size **1024** inside teini
 - TODO.txt File size **2048** inside teini
- server = Directory "www" added to root
 - .bashrc File size **512** inside server

Object diagram (write like this):

Objects (instances + attribute values):

- root:Directory {name="/"}
- home:Directory {name="home"}
- teini:Directory {name="teini"}
- server:Directory {name="www"}
- bashrc1:File {name=".bashrc", size=1024}
- todo:File {name="TODO.txt", size=2048}
- bashrc2:File {name=".bashrc", size=512}

Links (composition / contains):

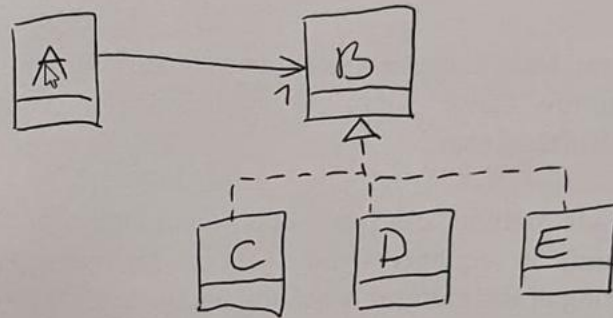
- root contains → home
- root contains → teini
- root contains → server
- teini contains → bashrc1

- `teini` contains $\rightarrow$ `todo`
- `server` contains $\rightarrow$ `bashrc2`

Check (as in test):

- `root.numberOfBytes()` = $1024 + 2048 + 512 = 3584$
- `root.numberOfFiles()` = 3

3) Given the following class diagram: [2 + 5 + 3]



- Which **design pattern** is this?
- Justify your decision by giving the classes **names** that explain the **elements of the design pattern**.
- Describe a **concrete use case** for this pattern (including a class diagram for the concrete example).

3) Pattern identification from diagram

Diagram meaning:

- **A** holds exactly **one** reference to **B** (multiplicity 1)
- **C, D, E** are subclasses/implementations of **B**

3(a) Which design pattern is this?

✓ **Strategy Pattern**

3(b) Justify by naming roles (pattern elements)

- **A = Context**

- **B = Strategy (interface / abstract class)**
- **C, D, E = ConcreteStrategy implementations**

So Context (A) uses a Strategy (B), and can swap C/D/E to change behavior.

3(c) Concrete use case (with class diagram idea)

Example: Payment method selection

- `Checkout` uses one payment strategy
- Payment can be done by card, PayPal, bank transfer

Roles:

- `Checkout` = Context
- `PaymentMethod` = Strategy
- `CreditCardPayment`, `PayPalPayment`, `BankTransferPayment` = ConcreteStrategies

Simple class diagram text:

- `Checkout` **has** `payment: PaymentMethod`
- `Checkout.pay(amount)` → **calls** `payment.pay(amount)`
- `PaymentMethod` **defines** `pay(amount)`
- Concrete classes implement `pay(amount)` differently

This matches exactly the given structure ($A \rightarrow B$ and B has C/D/E).

4) In the context of Message-Oriented Architectures (MOA): [3+7 points]

a) Which of the following statements accurately describe a **Message-Oriented Architecture (MOA)**? (one to four answers can be correct)

- i. In MOA, components communicate exclusively through the exchange of messages.
- ii. MOA primarily relies on synchronous remote procedure calls to exchange data.
- iii. Messages in MOA can represent commands, requests, or data payloads that trigger specific actions.
- iv. MOA avoids using middleware like message queues or brokers for communication.

b) Which of the following statements best describes the **communication structure** in a Message-Oriented Architecture (MOA)? (one to four answers can be correct)

- i. Components communicate by invoking each other's methods directly through synchronous function calls.
- ii. Components communicate exclusively by exchanging messages through middleware such as message queues or brokers.
- iii. Components share a common database where they read and write data to coordinate actions.
- iv. Components use remote procedure calls (RPC) to directly execute functions on other components over the network.

c) Describe a practical example of a MOA (including sketch).

4)

Message-Oriented Architectures (MOA)

4(a) Statements that describe MOA

Correct answers:

- ✓ **i.** Components communicate exclusively through exchanging messages.
- ✓ **iii.** Messages can be commands/requests/data payloads triggering actions.

Incorrect:

- **ii** is wrong (that is RPC / synchronous calls, not MOA focus)
- **iv** is wrong (MOA usually uses middleware like brokers/queues)

So: **i and iii**

4(b) Statements that describe MOA communication structure

Correct answer:

✅ **ii.** Components communicate exclusively by exchanging messages through middleware (queues/brokers).

Incorrect:

- i and iv are direct synchronous calls/RPC (not MOA)
- iii is shared database coordination (not message-oriented)

So: **ii only**

4(c) Practical example of MOA (with sketch)

Example: Online shop order processing (event-driven)

- OrderService sends message: **OrderCreated**
- PaymentService consumes it → sends **PaymentConfirmed**
- InventoryService consumes OrderCreated → sends **StockReserved**
- ShippingService consumes confirmations → ships order

Sketch (easy ASCII):

```
OrderService  -->  [Message Broker / Queue]  --> PaymentService
                                     |
                                     --> InventoryService
                                     |
                                     --> ShippingService
```

Why MOA:

- Services do **not call each other directly**
- They communicate using **messages**
- Loose coupling, scalable, asynchronous possible

5) In the context of class design principles [3 + 3 + 4 points]

- a) Which of the following statements best describes the **Single-Responsibility Principle (SRP)**? (one to four answers can be correct)
- i. A class should have only one reason to change, meaning it should focus on a single responsibility.
 - ii. A class can handle multiple responsibilities as long as those responsibilities are somewhat related.
 - iii. A class should be designed to incorporate as many functionalities as possible to reduce the total number of classes.
 - iv. A class's responsibilities should be tightly coupled to ensure that changes in one functionality automatically update the others.
- b) Which of the following statements best describe the **Open-Closed Principle (OCP)**? (one to four answers can be correct)
- i. Software entities (methods, classes, modules, etc.) should be open for extension, meaning their behavior can be extended to meet new requirements.
 - ii. Software entities must be modified whenever new behavior is added, ensuring they always reflect the latest requirements.
 - iii. Software entities should be closed for modification, so that extending their behavior does not involve altering the existing source or binary code.
 - iv. Open for extension means that a module's core functionality cannot be extended by adding new features without changing its original implementation.
- c) Which of the following statements accurately describe the **Dependency-Inversion Principle (DIP)**? (one to four answers can be correct)
- i. High-level modules should directly depend on low-level modules to minimize redundancy.
 - ii. Both high-level and low-level modules should depend on abstractions rather than concrete implementations.
 - iii. Abstractions should not depend on implementation details; instead, details should depend on abstractions.
 - iv. High-level modules should depend on concrete implementations to ensure reliable behavior.

5) Class design principles (SRP, OCP, DIP)

5(a) SRP

✓ Correct: i

- “One reason to change” = one responsibility
-

5(b) OCP

✓ Correct: **i and iii**

- Open for extension
 - Closed for modification
-

5(c) DIP

✓ Correct: **ii and iii**

- Both high-level and low-level depend on abstractions
- Details depend on abstractions, not the other way

Explanation of every Questions

Question 1:

Absolutely — I’ll teach you **Facade Pattern from zero**, then we’ll walk through **your Question 1 step-by-step**, and you’ll see *why* that answer is correct.

1) First: what is a “design pattern”?

A **design pattern** is a common, proven way to solve a repeated software design problem.

Like in electronics:

- If you need stable voltage → you often use a **regulator circuit**.
 - In software, if you want to simplify a complicated subsystem → you often use a **Facade**.
-

2) What is the Facade Pattern (simple meaning)

Facade = “front desk / reception”.

Imagine a hospital:

- You (customer) do NOT directly talk to:
 - lab department,
 - doctor department,
 - billing department,
 - pharmacy ...
- You go to the **reception desk**.
- Reception desk internally contacts all those departments in correct order.

✓ That reception desk is the **Facade**.

Definition (easy):

A **Facade** is a class that provides **one simple interface** to a **set of complex classes**.

3) Why do we use Facade? (Main reasons)

✓ 1) **Makes the system easier to use**

Instead of calling many objects, the client calls **one** object.

✓ 2) **Reduces coupling (dependency)**

If the subsystem changes, only the facade may need updating.
The client doesn't break.

✓ 3) **Cleaner code**

The client code becomes small and readable.

4) When is Facade useful?

When you have a “subsystem” with multiple classes and the client must do many steps, like:

- log something
- validate something
- calculate something

A facade can wrap all those steps into ONE method call.

Now let's explain your Question 1

What you are given (subsystem classes)

Your diagram has 3 classes:

1. **Validator**
 - `isValid()`
2. **Logger**
 - `debug()`
3. **Evaluator**
 - `sum(vector<double>)`

These three together are like “departments” inside a hospital.

But there is **NO easy single entry point** yet.

What the question asks in (a)

“Extend the class diagram to a Facade Pattern.

Add the “Calculator” facade class and the “Client”.

Add a method called `calcSum(vector<double>)` to Calculator.”

Meaning:

You must create:

- **Client** (the user of the system)
- **Calculator** (the reception desk/facade)
- Calculator hides the complexity: it talks to Logger, Validator, Evaluator.

So the Client should only call:

✓ `Calculator.calcSum(values)`

Instead of calling:

- `Logger.debug()`
- `Validator.isValid()`
- `Evaluator.sum()`

✓ **Correct solution for (a): What the class diagram should look like**

Classes:

- Client
- Calculator (Facade)
- Logger
- Validator
- Evaluator

Relationships:

- Client → Calculator (Client uses Calculator)
- Calculator → Logger (Calculator uses Logger)
- Calculator → Validator
- Calculator → Evaluator

In plain words:

Client does NOT talk to Logger/Validator/Evaluator.
Client talks only to **Calculator**.

That is the key idea of Facade.

Now Question (b): Sequence diagram

The question says:

“The facade method `calcSum(vector<double>)` calls `debug()`, `isValid()`, and `sum()` in sequence.”

So inside `Calculator.calcSum()`:

1. call `Logger.debug()`
2. call `Validator.isValid()`
3. call `Evaluator.sum(values)`
4. return result back to Client

✅ Sequence diagram (in easy written form)

You can draw it like this:

Client → **Calculator**: `calcSum(values)`

Calculator → **Logger**: `debug()`

Calculator → **Validator**: `isValid()` → returns true/false

Calculator → **Evaluator**: `sum(values)` → returns result

Calculator → **Client**: return result

That’s exactly what the question asked.

Why this is the correct answer (logic)

Because the question explicitly wants a **Facade Pattern**.

Facade pattern must have:

- ✅ a simplified class (Facade) that hides subsystem complexity
- ✅ a client that uses only the facade
- ✅ the facade internally uses subsystem classes

Here:

- Subsystem = `Logger`, `Validator`, `Evaluator`
- Facade = `Calculator`
- Client = `Client`

So the structure matches the formal Facade design.

Let's imagine real code (super easy)

This makes it very clear.

Without Facade (bad for client)

```
Logger log;
Validator val;
Evaluator eval;

log.debug();
if(val.isValid(values)) {
    result = eval.sum(values);
}
```

Client has to know everything.

With Facade (good)

```
Calculator calc;
result = calc.calcSum(values);
```

Inside Calculator:

```
double Calculator::calcSum(vector<double> v) {
    logger.debug();
    if(!validator.isValid(v)) return 0;
    return evaluator.sum(v);
}
```

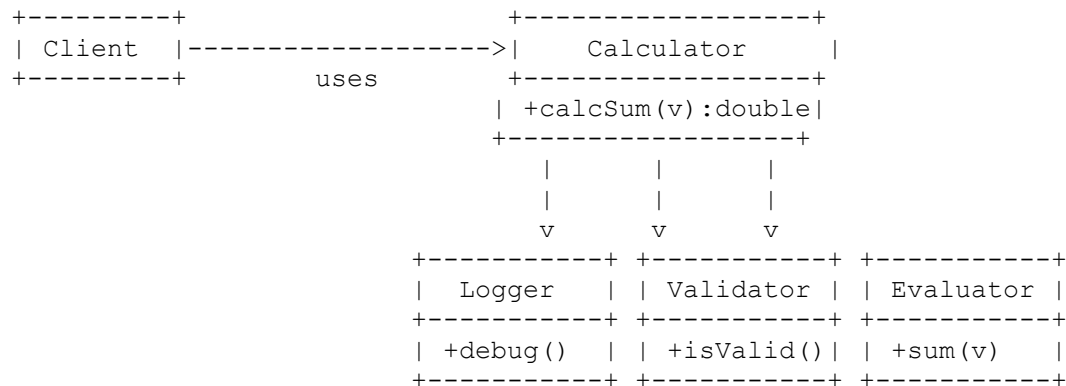
That is exactly the pattern.

Summary in 3 lines (you should remember)

- ✓ Facade = one simple class that hides many complex classes
 - ✓ Client uses only the Facade
 - ✓ Facade calls subsystem methods inside in the correct order
-

Here you go — neat **copy-friendly UML drawings (ASCII style)** for **Question 1(a) and 1(b)**.

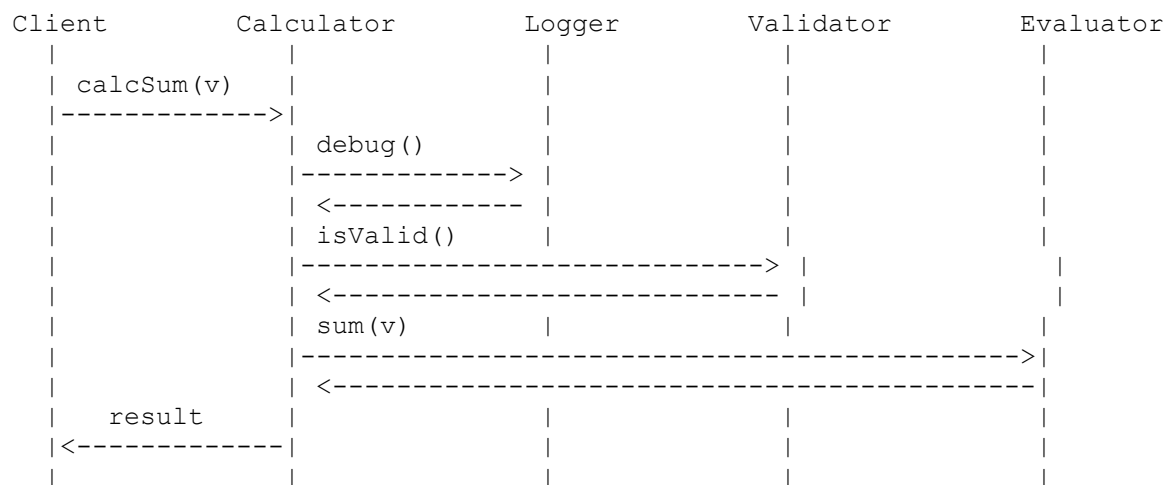
✓ Q1(a) Facade Pattern — Class Diagram (ASCII UML)



What this shows

- **Client only talks to Calculator**
- **Calculator is the Facade**
- Calculator internally uses **Logger, Validator, Evaluator**

✓ Q1(b) Sequence Diagram for `calcSum(vector<double>)`



Notes (for exam)

- The order is **exactly**: `debug()` → `isValid()` → `sum()`
- Finally Calculator returns the result to Client.

Question 2:

Got it — let's learn **Question 2 (Composite Pattern)** the same way as Q1: from **zero**, why we use it, and then **how to solve (a) and (b)** using your given test code.

1) What problem does Composite Pattern solve?

Sometimes we have things that are:

- **single items** (example: a file)
- **groups of items** (example: a folder/directory that contains files and other folders)

But we want to treat both **the same way**.

Real-life example

In a file system:

- A **File** has size (bytes)
- A **Directory** has size too (sum of everything inside it)

We want to call:

✓ `numberOfBytes()` on **both** a File and a Directory and get the correct result.

That idea = **Composite Pattern**.

2) Simple definition (easy)

Composite Pattern lets you build a **tree structure** where:

- **Leaf** = single object (File)
- **Composite** = object that contains other objects (Directory)
- Both share the same parent type/interface (Node)

So client can treat them uniformly:

- File is a Node
 - Directory is a Node
-

3) Understand your given classes

The diagram gives:

Node

- `name : string`
- `numberOfBytes()`

File

- `size : long`

Directory

- (empty in diagram, but will contain nodes)

So Node is the common type.

Directory and File must become “Node types”.

☒ Part (a): Insert missing relationships to form Composite Pattern

To become Composite, we need two key relationships:

(a1) Inheritance (Generalization)

File IS-A Node

Directory IS-A Node

So draw arrows:

- `File ----|> Node`
- `Directory ----|> Node`

(arrow pointing to Node)

(a2) Composition (Directory has children Nodes)

A directory can contain **many Nodes**, because inside a folder you can have:

- Files
- Other Directories

So we add:

- Directory **contains** 0..* Node

UML style:

Directory 1 ■—— 0..* Node

(black diamond means Directory owns them)

☒ What methods do in Composite?

For File

`numberOfBytes()` returns its own size:

- `File.numberOfBytes() = size`

For Directory

Directory must sum all children:

- `Directory.numberOfBytes() = sum(child.numberOfBytes())`

So Directory recursively adds sizes from all files in its tree.

☒ Part (b): Draw the Object Diagram using the Test Case

Now we use the code you showed.

Object diagram means: **instances** + **attribute values** + **links**.

Let's read the code like a story.

Step-by-step from code

1) Create root directory "/"

```
auto root = make_shared<Directory>("/");
```

So object:

- `root:Directory name="/"`
-

2) Create "home" directory and add to root

```
auto home = make_shared<Directory>("home");  
root->addNode(home);
```

So:

- `home:Directory name="home"`
 - `root` contains `home`
-

3) Create "teini" directory and add to root

```
auto teini = make_shared<Directory>("teini");  
root->addNode(teini);
```

So:

- `teini:Directory name="teini"`
 - `root` contains `teini`
-

4) Create two files inside teini

```
auto bashrc = make_shared<File>(".bashrc", 1024);  
teini->addNode(bashrc);
```

```
auto todo = make_shared<File>("TODO.txt", 2048);  
teini->addNode(todo);
```

Objects:

- bashrc1:File name=".bashrc", size=1024
 - todo:File name="TODO.txt", size=2048
- And links:
- teini contains bashrc1 and todo
-

5) Create "www" directory and add to root

```
auto server = make_shared<Directory>("www");
root->addNode(server);
```

Objects:

- server:Directory name="www"
- Link:
- root contains server
-

6) Add a file inside server

```
auto bashrc2 = make_shared<File>(".bashrc", 512);
server->addNode(bashrc2);
```

Object:

- bashrc2:File name=".bashrc", size=512
- Link:
- server contains bashrc2
-

☒ Final Object Diagram (you can copy in exam)

Objects (instances with attribute values)

- root:Directory {name="/"}
- home:Directory {name="home"}
- teini:Directory {name="teini"}
- server:Directory {name="www"}
- bashrc1:File {name=".bashrc", size=1024}
- todo:File {name="TODO.txt", size=2048}
- bashrc2:File {name=".bashrc", size=512}

Links (containment)

- root contains → home
 - root contains → teini
 - root contains → server
 - teini contains → bashrc1, todo
 - server contains → bashrc2
-

Why the verification lines work (very important)

```
EXPECT_EQ(1024 + 2048 + 512, root->numberOfBytes());  
EXPECT_EQ(3, root->numberOfFiles());
```

numberOfBytes

Root contains 3 directories, but only files have sizes:

- inside teini: 1024 + 2048
- inside www: 512

Total:

 $1024 + 2048 + 512 = 3584$

So root.numberOfBytes() must recursively sum everything.

numberOfFiles

Files are:

- bashrc1
- todo
- bashrc2

Total:

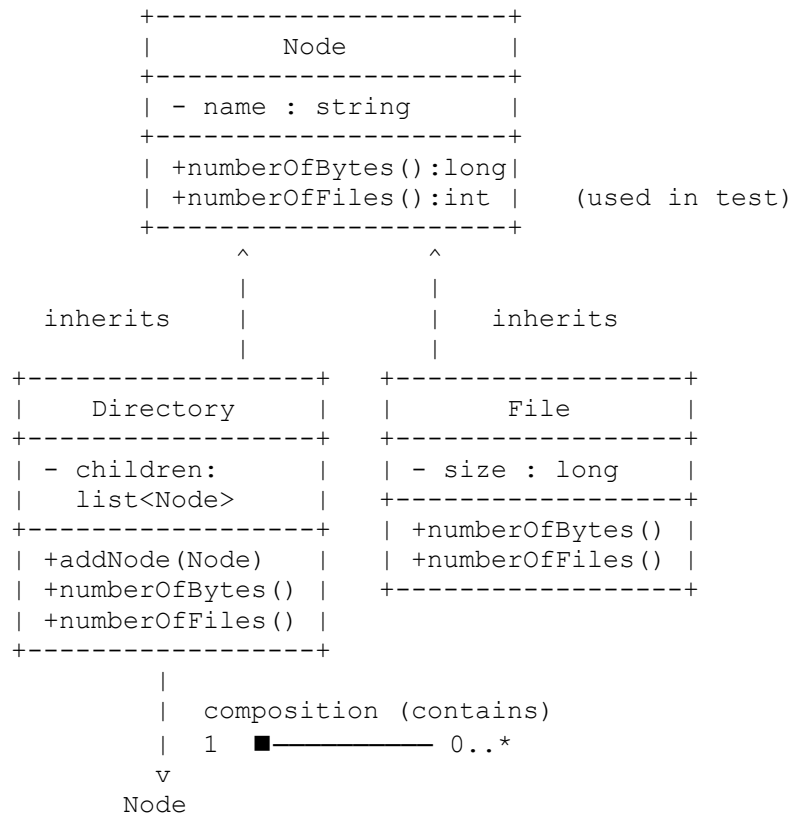
 3 files

So root.numberOfFiles() is also recursive.

One-sentence memory trick

 **Composite pattern = Tree structure** where Directory (composite) contains Nodes, File (leaf) is also a Node, and both share same operations like `numberOfBytes()`.

✓ Q2(a) Composite Pattern — Class Diagram (ASCII UML)



What to remember

- **File IS-A Node**
- **Directory IS-A Node**
- **Directory HAS many Nodes** (files or directories)

✓ Q2(b) Object Diagram — From the given test case

Objects with attribute values

```

root    :Directory {name="/"}
home    :Directory {name="home"}
teini   :Directory {name="teini"}
server  :Directory {name="www"}

bashrc1:File {name=".bashrc", size=1024}
todo    :File {name="TODO.txt", size=2048}

```

```
bashrc2:File {name=".bashrc", size=512}
```

Containment links (who contains what)

```
root ■—contains—> home
root ■—contains—> teini
root ■—contains—> server
```

```
teini ■—contains—> bashrc1
teini ■—contains—> todo
```

```
server■—contains—> bashrc2
```

Quick verification (matches the test)

- `root.numberOfBytes()` = $1024 + 2048 + 512 = 3584$
- `root.numberOfFiles()` = 3 (bashrc1, todo, bashrc2)

Nice — here's the **tree picture (folder tree style)** for your Composite example. This is super easy to remember and matches the test case exactly:

```
/ (root)
├── home/
├── teini/
│   ├── .bashrc    (1024 bytes)
│   └── TODO.txt   (2048 bytes)
└── www/
    └── .bashrc    (512 bytes)
```

How this maps to the Composite Pattern

- `/, home, teini, www` → **Directory (Composite)**
- `.bashrc, TODO.txt` → **File (Leaf)**
- All of them are **Node**

Why this helps in exams

- You can **draw this tree** quickly.
- Then convert it into:
 - the **object diagram** (instances + links), and
 - the **calculation** for `numberOfBytes()` and `numberOfFiles()`:
 - $\text{Bytes} = 1024 + 2048 + 512 = 3584$
 - $\text{Files} = 3$

Question 3:

Sure — let's learn **Question 3** from absolute basics (like we did for Facade + Composite). I'll explain **what the diagram means, which pattern it is, why**, and how to answer **(a), (b), (c)** in an exam.

1) First read the diagram (what does it show?)

You see 5 classes: A, B, C, D, E

- $A \rightarrow B$ (1)
This means: **A has exactly ONE B** (A uses one B object).
- $C, D, E \rightarrow B$ (inheritance/implementation)
This means: **C, D, E are different versions of B**
(B is an interface/abstract class, and C/D/E are concrete implementations)

So the meaning is:

- ✓ A can work with any of (C or D or E) **through B**.

2) What design pattern is this?

- ✓ **Strategy Pattern**

Because Strategy Pattern is exactly:

- A = Context
- B = Strategy interface
- C/D/E = Concrete strategies

A contains a reference to a Strategy (B), and you can swap the concrete one (C/D/E).

3) Strategy Pattern from zero (easy meaning)

Real-life example:

You want to travel from home to school.

You can choose:

- by car
- by bus
- by bike

The goal is the same: **travel()**

But the algorithm differs.

So:

- “Travel method” is the **Strategy**
 - CarTravel / BusTravel / BikeTravel are **ConcreteStrategies**
 - You (or an app) uses one strategy at a time = **Context**
-

4) Why do we use Strategy Pattern?

✓ Main reason:

To change behavior/algorithm **without changing the main class**.

Benefits:

- avoids lots of `if/else` or `switch` inside A
 - easy to add new behavior later (add new class F, no need to change A)
 - clean, extensible design
-

Now answer your Question 3 exactly

✓ 3(a) Which design pattern is this?

Answer: Strategy Pattern

✓ 3(b) Justify with role names (pattern elements)

You must rename A, B, C, D, E according to Strategy pattern roles:

- **A = Context**
- **B = Strategy (interface / abstract class)**
- **C = ConcreteStrategy1**
- **D = ConcreteStrategy2**
- **E = ConcreteStrategy3**

That is the justification.

Also mention:

- A has exactly one Strategy (B)
 - C/D/E implement B
-

✓ 3(c) Concrete use case + class diagram (example)

Example (easy + common): Payment system

You buy something and pay with different methods:

- Credit Card
- PayPal
- Bank Transfer

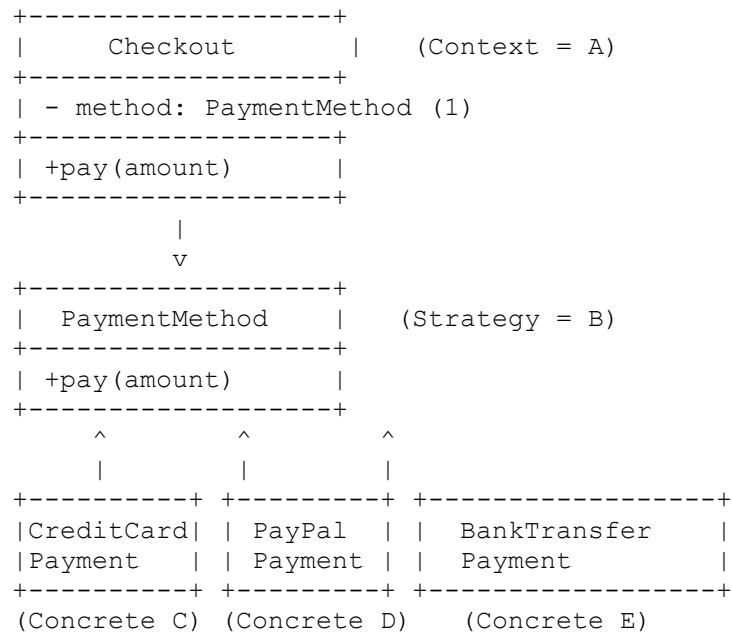
Same operation: `pay(amount)`

But implementation differs.

Mapping to pattern roles:

- **Context (A)** → `Checkout`
 - **Strategy (B)** → `PaymentMethod`
 - **ConcreteStrategies (C/D/E)** →
 - `CreditCardPayment`
 - `PayPalPayment`
 - `BankTransferPayment`
-

Exam-ready class diagram (ASCII UML)



Explanation (short for exam)

Checkout uses a PaymentMethod strategy.

You can change payment behavior by changing the PaymentMethod object.

Super simple memory trick

✅ **Strategy = “one interface, many algorithms”**

Context holds 1 strategy and can swap it.

Question 4:

Sure — let's learn **Question 4 (MOA)** from basics and then I'll explain **exactly why the correct options are correct** and why the others are wrong.

1) What is MOA (Message-Oriented Architecture)?

MOA means:

✅ **Components/services communicate by sending messages**, not by calling each other's functions directly.

Think of MOA like **WhatsApp**:

- You don't "run" your friend's function.
- You **send a message**.
- Your friend reads it later and replies.

So MOA is often:

- **asynchronous**
- **decoupled**
- uses **middleware** like:
 - message broker
 - message queue (RabbitMQ, Kafka, etc.)

2) What is a "message" here?

A message can be:

- a command: "ReserveStock"
- an event: "OrderCreated"
- data payload: {orderId: 5, total: 100€}

So messages are not just text — they are **structured data**.

Now answer (a)

4(a) Which statements describe MOA?

✓ (i) Correct

“In MOA, components communicate exclusively through the exchange of messages.”

Why correct:

That is literally the definition of message-oriented communication.
No direct method call — only messages.

✗ (ii) Wrong

“MOA primarily relies on synchronous remote procedure calls”

Why wrong:

RPC = calling a function on another system directly (like HTTP request expecting immediate response).
That is *not* message-based architecture.
MOA focuses on **messages**, often **async**, via broker/queue.

✓ (iii) Correct

“Messages can represent commands, requests, or payloads that trigger actions.”

Why correct:

In MOA, a message often tells a service:

- “do something”
or says:
- “something happened” (event)

So yes, messages trigger actions.

✗ (iv) Wrong

“MOA avoids using middleware like queues/brokers”

Why wrong:

MOA usually **depends** on middleware:

- message broker (Kafka / RabbitMQ)
- queues

That middleware is the core of MOA.

✅ So **correct answers for (a): i and iii**

Now answer (b)

4(b) Which describes the communication structure in MOA?

❌ (i) Wrong

“Components invoke each other’s methods directly through synchronous function calls.”

That is direct coupling + synchronous calls.
This is not MOA.

✅ (ii) Correct

“Components communicate exclusively by exchanging messages through middleware such as message queues or brokers.”

Why correct:

This is exactly MOA structure:

- sender puts message in broker/queue
 - receiver reads message later
 - they are not calling each other directly
-

❌ (iii) Wrong

“Components share a common database...”

That is called **database integration / shared DB architecture**, not MOA.
MOA uses messaging, not shared database as main communication.

✗ (iv) Wrong

“Components use RPC to directly execute functions...”

Again RPC = direct execution request, not message-based.

✓ So **correct answer for (b): ii only**

Now (c): example + sketch (why it matches MOA)

MOA Example: Online Shop Order System

When an order is created:

1. `OrderService` sends message: **OrderCreated**
2. `PaymentService` receives it and processes payment
3. `InventoryService` receives it and reserves items
4. `ShippingService` receives messages and ships later

Sketch

```
OrderService  --->  [Message Broker / Queue]  --->  PaymentService
                                     |
                                     --->  InventoryService
                                     |
                                     --->  ShippingService
```

Why this is MOA

- Services do not call each other directly
 - They communicate only through messages
 - Broker/queue is middleware
-

Easy memory rule for MCQ

✅ If you see: **message broker / queue / publish-subscribe / asynchronous messages** → that's MOA

❌ If you see: **direct call / RPC / synchronous method call** → not MOA

Question 5:

Sure — this question is about **SOLID class design principles** (SRP, OCP, DIP). I'll teach each one from zero, then show **why the correct options are correct** and why the others are wrong.

5(a) SRP — Single Responsibility Principle

What SRP means (very simple)

✅ **One class = one job**

If a class has many jobs, it will change for many reasons → harder to maintain.

Official easy line:

A class should have **only one reason to change**.

Example

Bad (one class does too much):

- calculates salary
- prints salary report
- saves to database

Good:

- SalaryCalculator
 - ReportPrinter
 - SalaryRepository
-

Now check the options (SRP)

✅ (i) **Correct**

A class should have only one reason to change...

This is exactly SRP.

❌ (ii) **Wrong**

A class can handle multiple responsibilities as long as related.

Even if related, it's still multiple responsibilities → breaks SRP.

❌ (iii) **Wrong**

Put as many functionalities as possible to reduce number of classes.

This makes “God classes” (huge classes) and breaks SRP.

❌ (iv) **Wrong**

Responsibilities should be tightly coupled so change updates others.

SRP prefers **low coupling**, not tight coupling.

✅ **Correct answer for 5(a): i**

5(b) OCP — Open/Closed Principle

What OCP means (very simple)

✅ You should be able to **add new features** without **changing existing working code**.

Easy sentence:

Open for extension, closed for modification.

Example

If you have payment types:

- Card
- PayPal

Tomorrow you add:

- Apple Pay

With OCP, you add a new class `ApplePayPayment` without editing old code.

Now check the options (OCP)

✓ (i) Correct

Software entities should be open for extension...

Yes: you should extend behavior for new requirements.

✗ (ii) Wrong

Must be modified whenever new behavior is added...

Opposite of OCP. OCP tries to avoid modifying stable code.

✓ (iii) Correct

Closed for modification so extending does not alter existing code...

Yes: this is the “closed for modification” part.

✗ (iv) Wrong

Open for extension means core functionality cannot be extended without changing implementation...

This contradicts OCP. OCP says you *can* extend without changing original.

✓ Correct answer for 5(b): i and iii

5(c) DIP — Dependency Inversion Principle

What DIP means (very simple)

✓ High-level code should not depend on low-level details.
Both should depend on **abstractions** (interfaces).

Easy wording

Depend on interfaces, not concrete classes.

Example

Bad:

- `OrderService` **directly uses** `MySQLDatabase`

Good:

- `OrderService` **uses** `Database` **interface**
- `MySQLDatabase` **implements** `Database`

So you can swap MySQL with PostgreSQL without changing `OrderService`.

Now check the options (DIP)

✗ (i) Wrong

High-level modules should directly depend on low-level modules...

Opposite of DIP.

✓ (ii) Correct

Both high-level and low-level modules should depend on abstractions...

Exactly DIP.

✓ (iii) Correct

Abstractions should not depend on details; details depend on abstractions.

This is the core sentence of DIP.

✗ (iv) Wrong

High-level modules should depend on concrete implementations...

Opposite of DIP.

✓ **Correct answer for 5(c): ii and iii**

Super easy memory cheat sheet

SRP

- ✓ One class = one responsibility
- ✓ One reason to change

OCP

- ✓ Add new features by extension
- ✓ Don't modify old working code

DIP

- ✓ Depend on interfaces (abstractions)
- ✓ Details depend on abstractions